shown in the following listing. If you chose to create an MSTest (or other framework) test project, the xUnit and xUnit runner packages would be replaced by packages appropriate to your testing framework of choice.

Listing 35.1 The .csproj file for an xUnit test project

```
<Project Sdk="Microsoft.NET.Sdk">                                    ❶
  <PropertyGroup>                                                    ❶
    <TargetFramework>net7.0</TargetFramework>                        ❶
    <IsPackable>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference
        Include="Microsoft.NET.Test.Sdk" Version="17.3.2" />         ❷
    <PackageReference Include="xunit" Version="2.4.2" />             ❸
    <PackageReference
        Include="xunit.runner.visualstudio" Version="2.4.5" />       ❹
    <PackageReference Include="coverlet.collector" Version="3.1.2" /> ❺
  </ItemGroup>
</Project>
```

❶ The test project is a standard .NET 7.0 project.

❷ The .NET Test SDK, required by all test projects

❸ The xUnit test framework

❹ The xUnit test adapter for the .NET Test SDK

❺ An optional package that collects metrics about how much of your code base is covered by tests

TIP Adding the Microsoft.NET.Test.Sdk package marks the project as a test project by setting the `IsTestProject` MsBuild property.

In addition to the NuGet packages, the template includes a single example unit test. This doesn't do anything, but it's a valid xUnit test all the same, as shown in the following listing. In xUnit, a test is a method on a public class, decorated with a `[Fact]` attribute.

Listing 35.2 An example xUnit unit test, created by the default template

```
public class UnitTest1        ❶
{
```